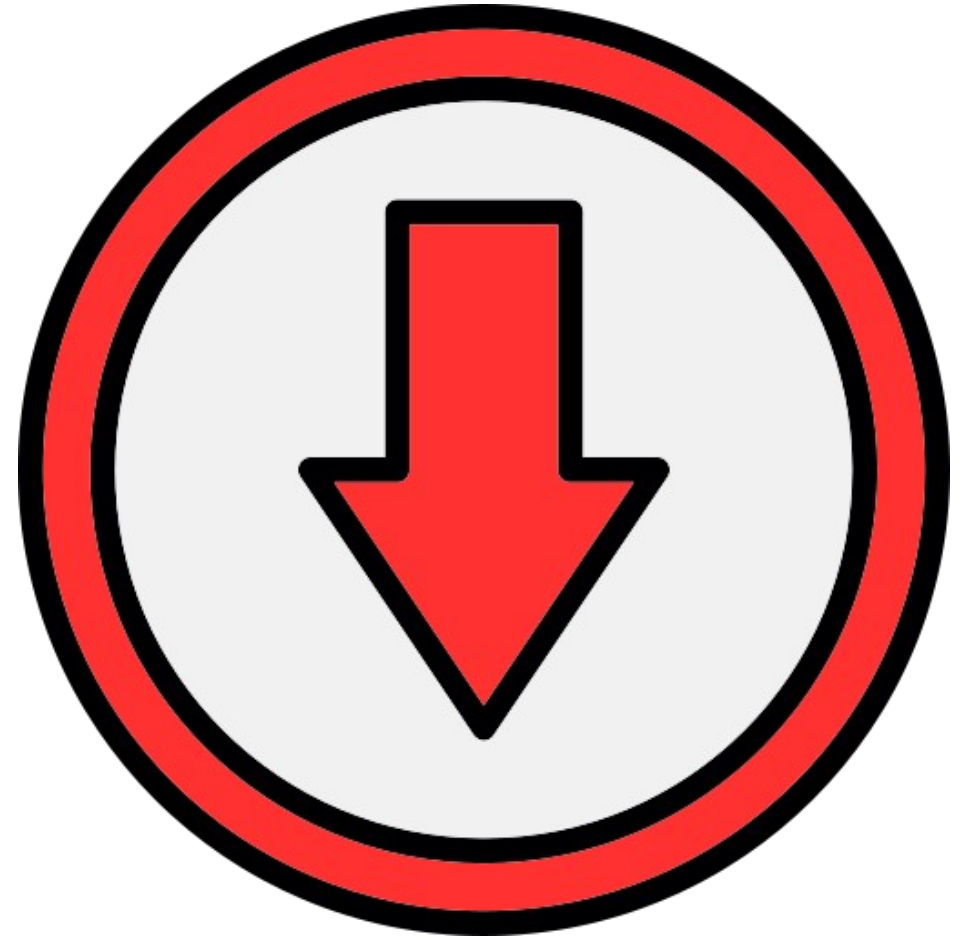


Sistemas Operativos de Tiempo Real (4)

Técnicas Digitales III

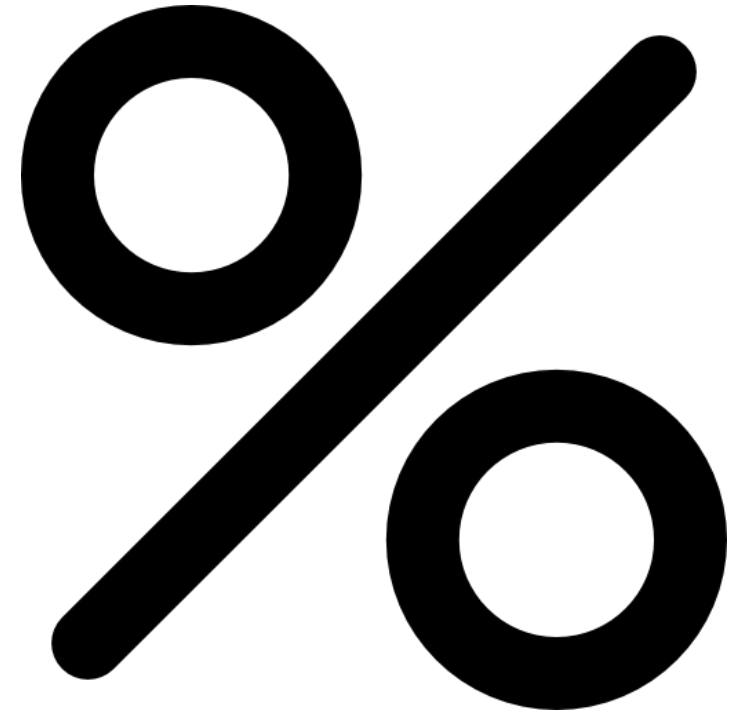
Tarea ociosa

- Ejecutar funciones de procesamiento continuo, en segundo plano o de baja prioridad sin la sobrecarga de RAM que supone la creación de tareas de aplicación para tal fin.



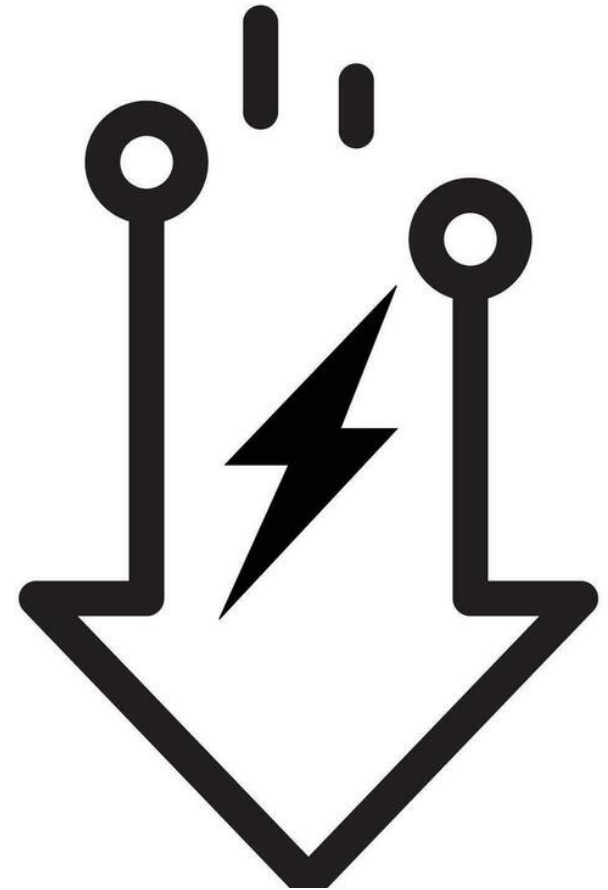
Tarea ociosa

- Medir la cantidad de capacidad de procesamiento disponible. (La tarea inactiva solo se ejecutará cuando todas las tareas de aplicación de mayor prioridad no tengan trabajo que realizar; por lo tanto, medir el tiempo de procesamiento asignado a la tarea inactiva proporciona una indicación clara del tiempo de procesamiento disponible).



Tarea ociosa

- Poner el procesador en modo de bajo consumo, lo que proporciona un método sencillo y automático para ahorrar energía cuando no hay procesamiento de aplicaciones que realizar.



Tarea ociosa

- Una función Idle Hook nunca debe intentar bloquearse ni suspenderse.
- *Nota: Bloquear la tarea inactiva de cualquier forma podría provocar que ninguna tarea esté disponible para entrar en el estado de ejecución.*

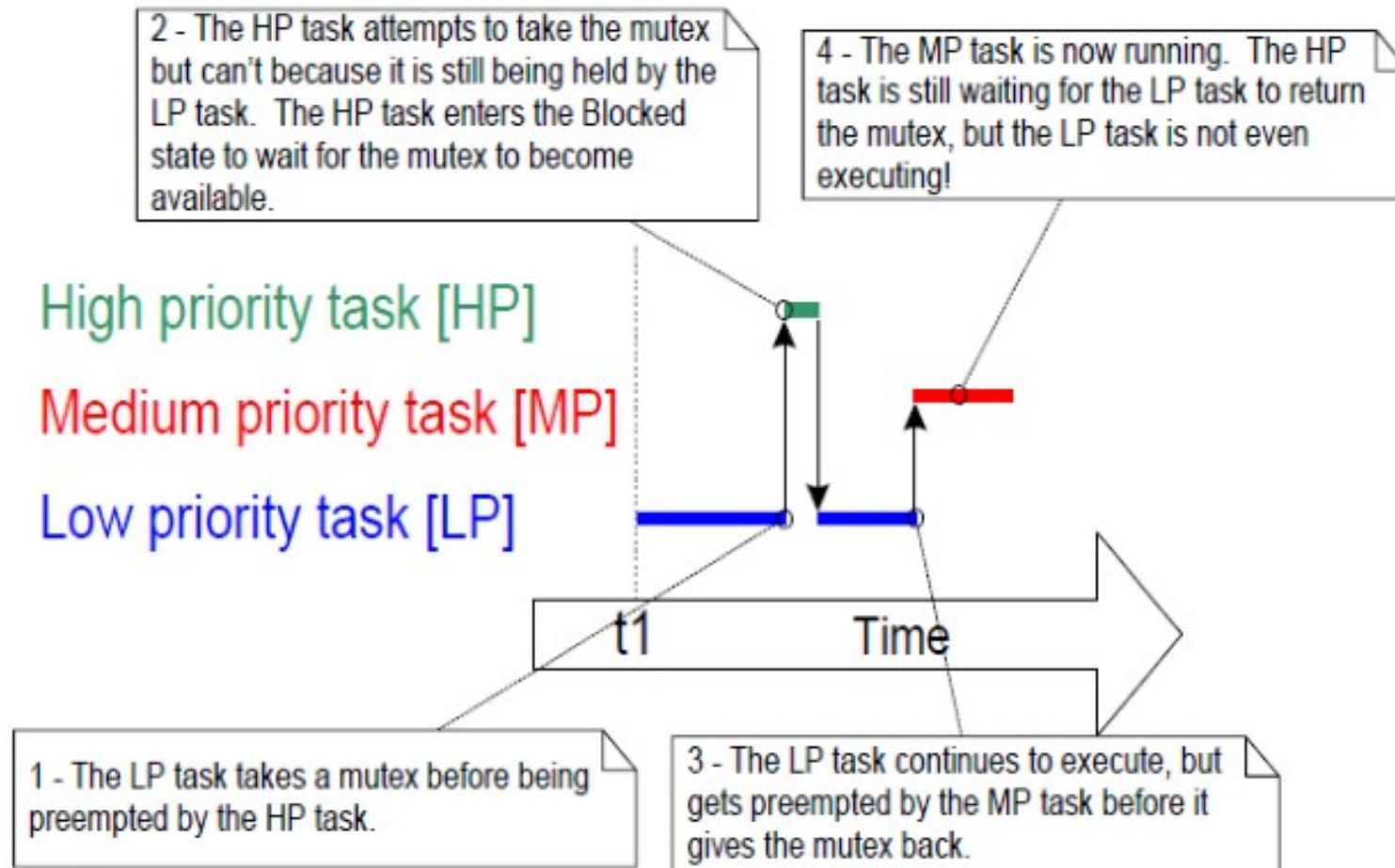


Tarea ociosa

- Si una tarea de la aplicación utiliza la función API `vTaskDelete()` para eliminarse, el Idle Hook debe siempre devolver el control a quien la llamó en un plazo razonable.
- Esto se debe a que la tarea inactiva es responsable de liberar los recursos del kernel asignados a las tareas que se eliminan.

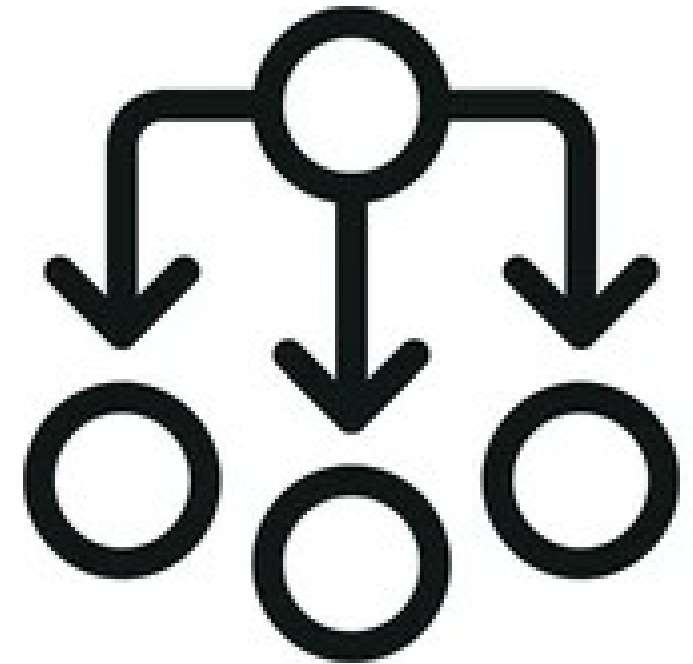


Inversión de prioridad+



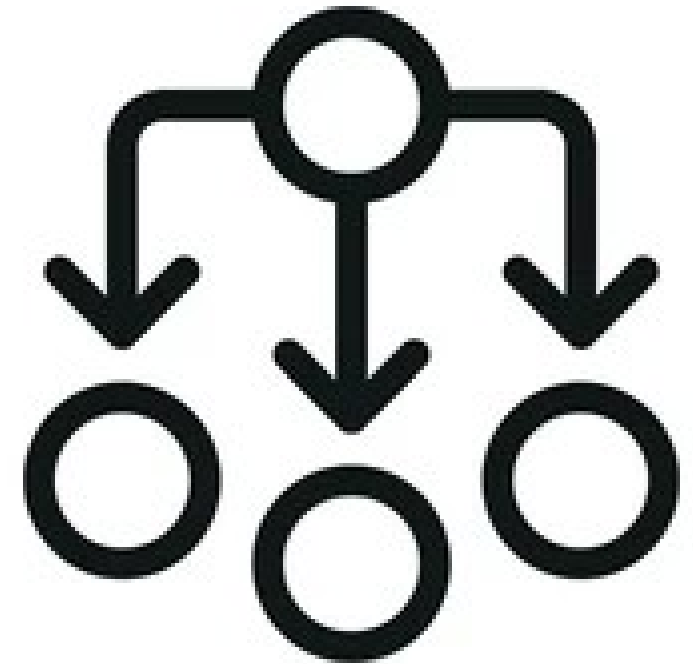
Inversión de prioridad+

- Los mutex y los semáforos binarios de FreeRTOS son muy similares.
- La diferencia radica en que los mutex incluyen un mecanismo básico de "herencia de prioridad", mientras que los semáforos binarios no.
- La herencia de prioridad es un esquema que minimiza los efectos negativos de la inversión de prioridad.



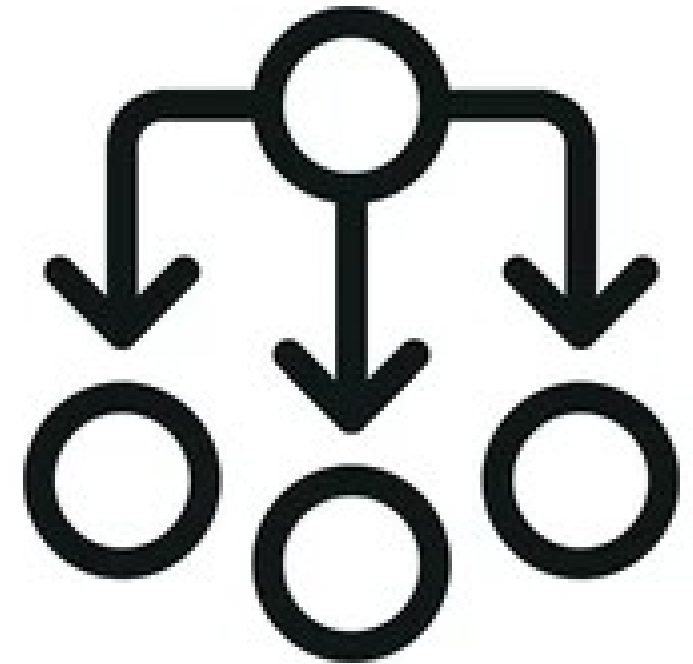
Inversión de prioridad+

- La herencia no corrige el problema, sino que simplemente reduce su impacto al garantizar que la inversión siempre esté limitada en el tiempo.
- De todas maneras, la herencia de prioridad complica el análisis de la temporización del sistema y no es recomendable depender de ella para su correcto funcionamiento.

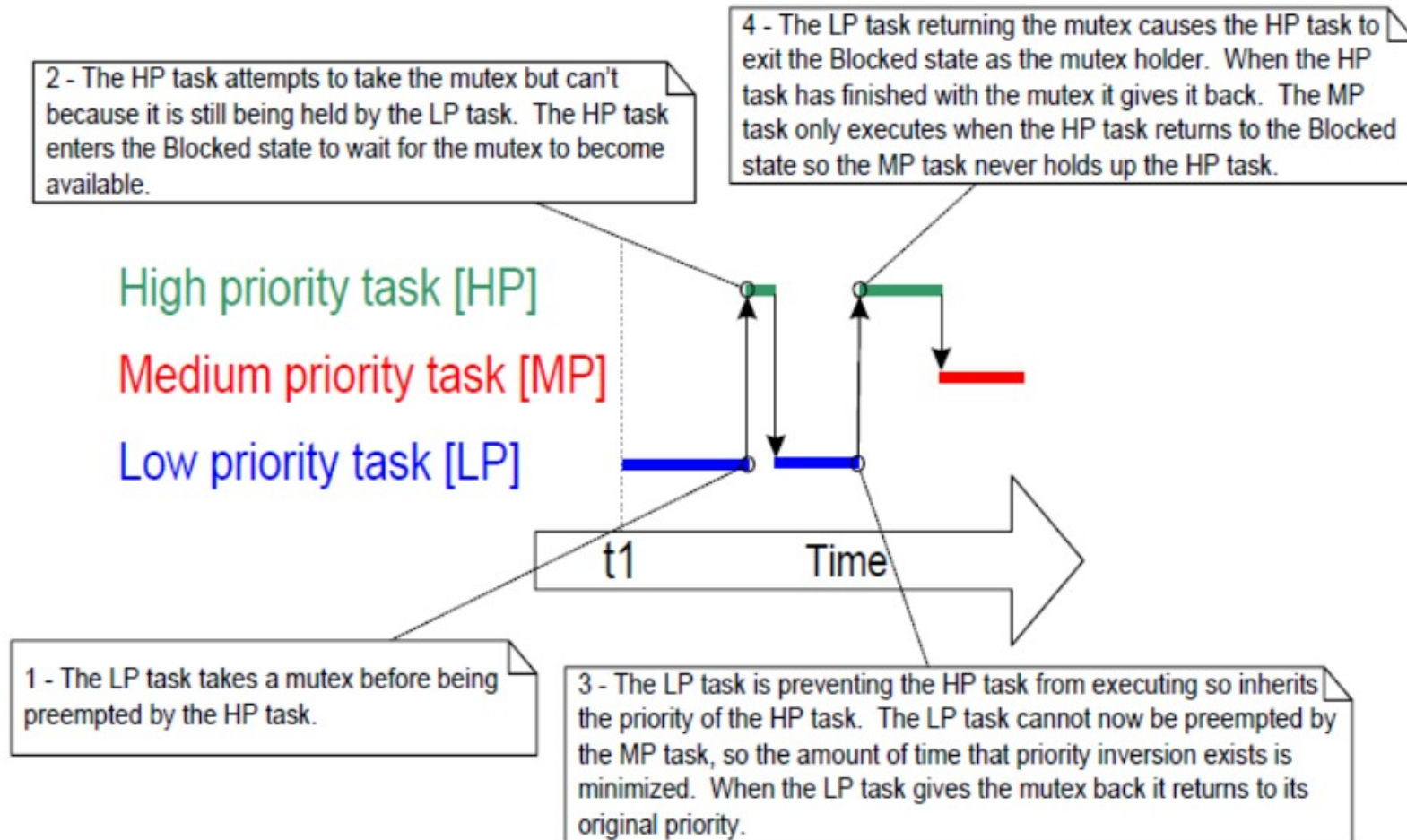


Inversión de prioridad+

- La herencia de prioridad funciona elevando temporalmente la prioridad del poseedor del mutex a la prioridad de la tarea de mayor prioridad que intenta obtener el mismo mutex.
- La tarea de menor prioridad que posee el mutex «hereda» la prioridad de la tarea que espera el mutex.



Inversión de prioridad+



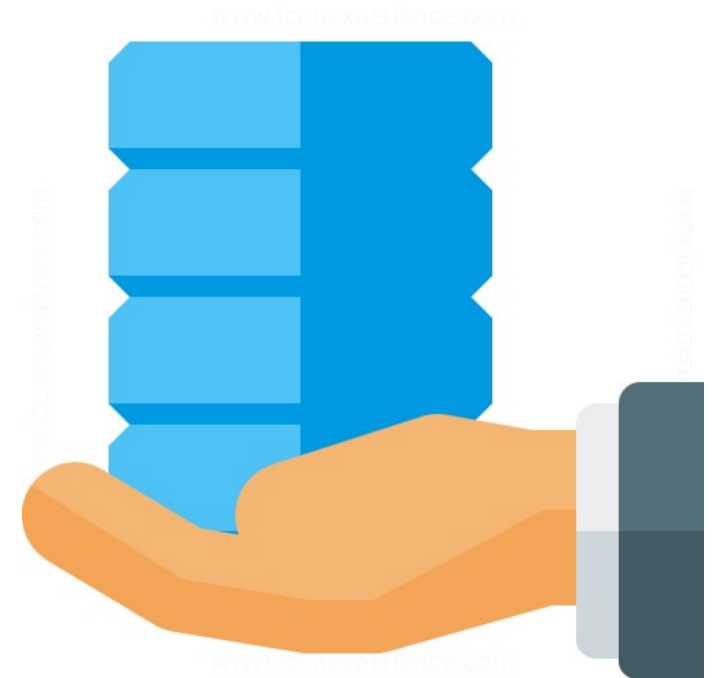
Deadlock+

- Tarea A toma Mutex 1 y espera Mutex 2.
- Tarea B toma Mutex 2 y espera Mutex 1.
- Resultado: Ambas tareas quedan congeladas en estado Blocked para siempre.



Tarea de control de acceso (Gatekeeper)

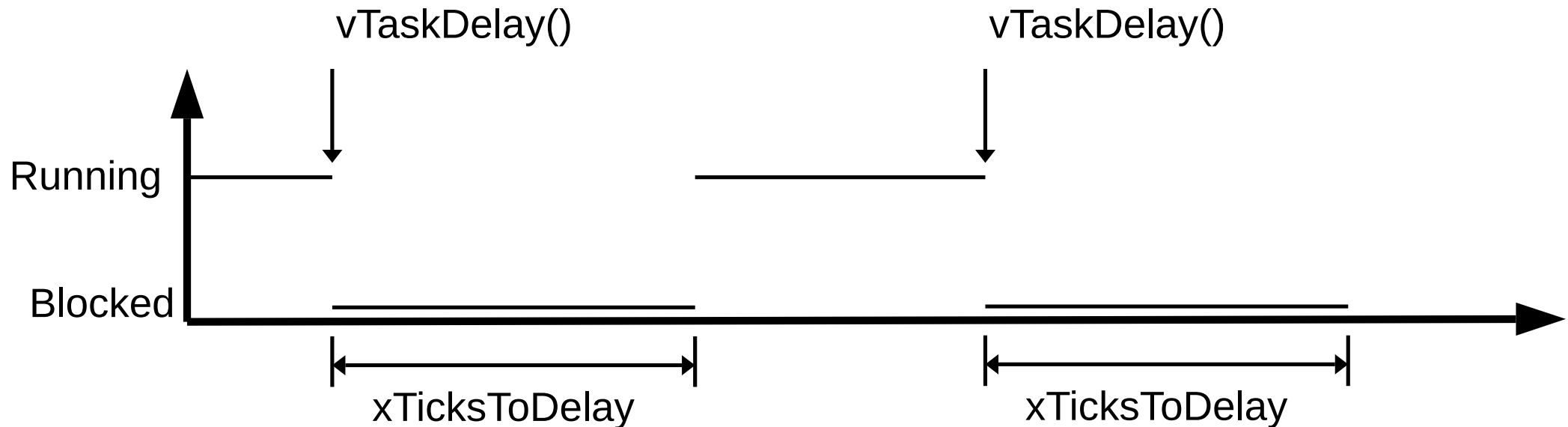
- Las tareas de control de acceso proporcionan un método limpio para implementar la exclusión mutua sin el riesgo de inversión de prioridad ni de interbloqueo.
- Una tarea de control de acceso es aquella que posee exclusivamente un recurso. Solo la tarea de control de acceso puede acceder directamente al recurso; cualquier otra tarea que necesite acceder a él solo puede hacerlo indirectamente, utilizando los servicios de la tarea de control de acceso.



vTaskDelay

vTaskDelay bloquea la tarea un tiempo determinado por su argumento, xTicksToDelay.

El tiempo que permanece bloqueada es absoluto, independiente del momento en el que se llame a la función.



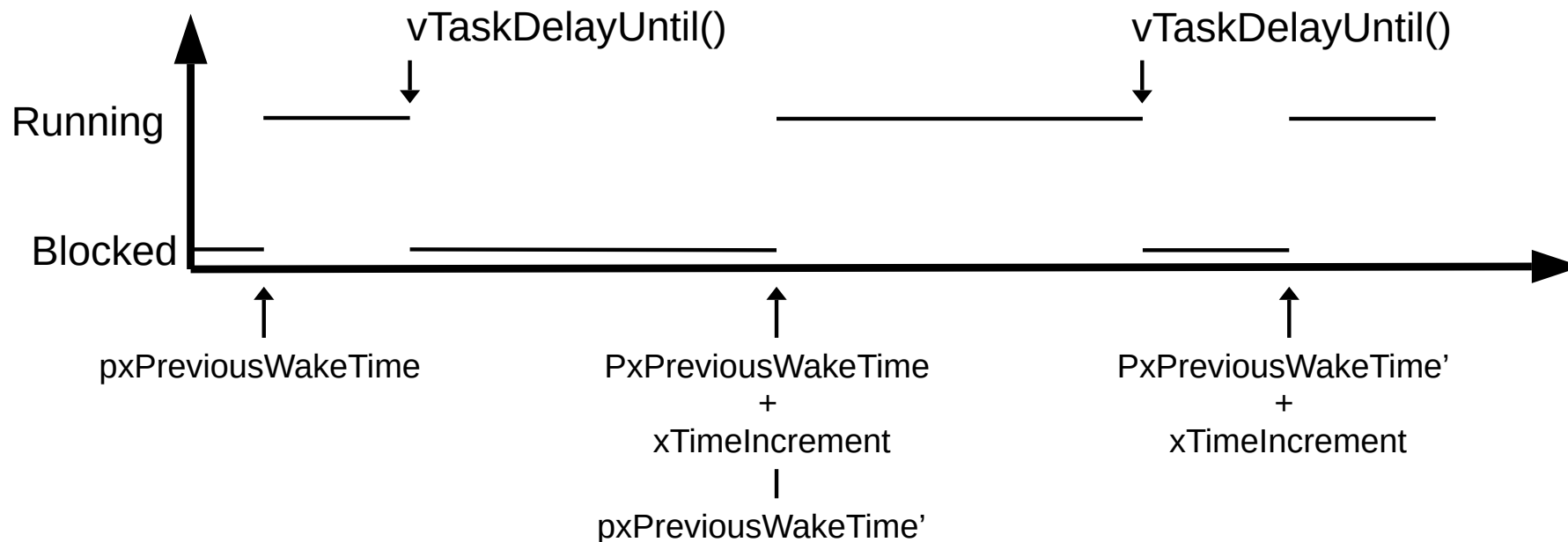
vTaskDelayUntil

- Los parámetros de vTaskDelayUntil() especifican el valor exacto del contador de ticks en el que la tarea que realiza la llamada debe pasar del estado Bloqueada al estado Lista.
- Esta función se debe usar cuando se requiere un período de ejecución fijo, ya que el momento en que se desbloquea la tarea que realiza la llamada es absoluto, en lugar de relativo al momento en que se llamó a la función (como ocurre con vTaskDelay()).

```
void vTaskDelayUntil( TickType_t * pxPreviousWakeTime,  
                    TickType_t xTimeIncrement );
```

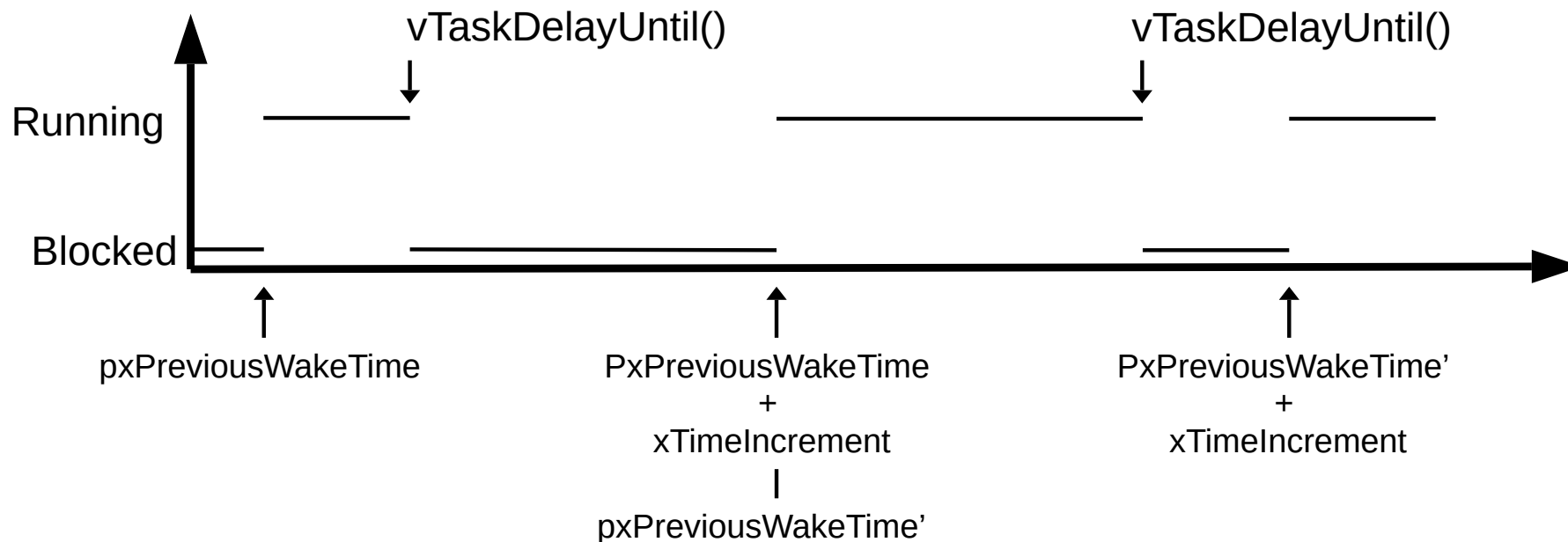
vTaskDelayUntil

- vTaskDelayUntil bloquea la tarea hasta que los ticks del sistema alcanzan el valor $pxPreviousWakeTime + xTimeIncrement$
- $pxPreviousWakeTime$ se debe actualizar cuando una tarea pasa al estado Running



vTaskDelayUntil

- De esta manera el tiempo que la tarea permanece bloqueada es variable, pasando al estado Running cada xTimeIncrement



xTicksToWait

```
BaseType_t xQueueReceive( QueueHandle_t xQueue,  
                          void * const pvBuffer,  
                          TickType_t xTicksToWait );
```

- El tiempo máximo que la tarea debe permanecer en estado Bloqueado para esperar a que haya datos disponibles en la cola, si esta ya está vacía.

xTicksToWait

```
for( ;; )
{
    /* Wait for the maximum period for data to become available on the queue.
    The period will be indefinite if INCLUDE_vTaskSuspend is set to 1 in
    FreeRTOSConfig.h. */
    if( xQueueReceive( xQueue, &xMessage, portMAX_DELAY ) != pdPASS )
    {
        /* Nothing was received from the queue - even after blocking to wait
        for data to arrive. */
    }
    else
    {
        /* xMessage now contains the received data. */
    }
}
```